Agentic LLM
Assignment 2: Reasoning

Leiden University

June 2, 2026

1 Introduction

In this assignment you will gain experience with reasoning and self-reflection.

- You will run reasoning LLMs on a puzzle-and-games benchmark, named BALROG, benchmarking their performance
- You will adapt your own reasoning agent and test its performance
- You will try out ideas for self-reflection in this agent

In all these exercises we invite you to critically reflect on the performance and the stability of agentic LLMs in puzzles and games.

2 Preparation and Setup

2.1 Preparation

- Read Parts 2 and 3 of the book (Chapter 6-9). This assignment text is based on Chapter 10 of the book.
- Go to the BALROG website `https://balrogai.com` and familiarize yourself. Read the paper [Paglieri et al., 2024].

BALROG is designed to challenge in-context agents on reasoning problems. It uses puzzle-style games to benchmark the reasoning capabilities of LLMs and VLMs. The agent determines the next step in the puzzle in each iteration.

The games are from easy to hard: BabyAI, Crafter, TextWorld, BabaIsAI, MiniHack, and NetHack. The book (Chapter 10) and the paper contain short descriptions of the benchmarks.

```
export OPENAI_API_KEY=<KEY>
export ANTHROPIC_API_KEY=<KEY>
export GEMINI_API_KEY=<KEY>
```

Listing 1: Installing the API key

2.2 Warning

Note that some of the benchmarks are very hard. For example, no LLM that we know off is expected to be able to solve NetHack. You are not expected to achieve the impossible, you are expected to try, to fail, to learn, and to reflect on what you learned. If you happen to have a hyper-competitive streak, be careful, do not waste too much compute on chasing the impossible. Follow a careful, gentle, step-by-step (where did we see that?), reasoning approach instead. We value the scientific approach, not blind ambition.

2.3 Setup

Go to `https://balrogai.com`, or directly to the Github repository at `https://github.com/balrog-ai/BALROG`. Install BALROG in your system with the `git clone` command. Install BALROG on your own computer.

Please experiment both with LLMs that are hosted in the cloud (such as GPT and Claude) and also with open LLMs that you host locally (such as Qwen and Ollama). Consider using local hosting tools such as LM Studio. Do not spend too much of your own money on commercial LLMs, that is not the goal of this assignment. Try using free student tiers.

The BALROG readme on the Github repository contains information in how to install the system, and how to install an API key for your favorite LLM, see Listing 1. (In the exercises we will use multiple LLMs, to compare their performance. For now, we are just trying to get the system to run, and any one will do.)

Please read the Agent tutorial in BALROG at `https://github.com/balrog-ai/BALROG/blob/main/docs/agents.md`. In BALROG, agents are small pieces of Python code that call an LLM with a prompt to provide an answer. The agent code is responsible for receiving answers from the model (observations) and for selecting the next action. The LLM is used to execute various prompts.

The agent code in BALROG queries the LLM: agents send observations to the LLM, receive responses, and use those responses to decide on actions.

BALROG comes with a number of pre-written agents, which you should study to design your own. The repository has a naive agent and a chain of thought agent. Also provided are more robust versions for handling LLM responses.

When you build your own agent, you will most likely build on the custom agent. The BALROG agent tutorial provides an example for a simple planning agent.

```
python eval.py \
 agent.type=naive \
 agent.max_image_history=0 \
 agent.max_text_history=16 \
 eval.num_workers=16 \
 client.client_name=openai \
 client.model_id=gpt-4o-mini-2024-07-18
```

Listing 2: Running the standard evaluation

3 Exercise 1: Running the Benchmark

We start by trying to reproduce the results in the BALROG paper. Please have a look at the BALROG code at Github `https://github.com/balrog-ai/BALROG`. Read the Evaluation Guide, the Agent Development tutorial, and the instructions how to run Few Shot Learning experiments (in-context learning). Install the relevant tools on your computer, and the right Python version (and put it in your PATH). Check the version by starting the Python interpreter from the command prompt: `$ python`.

First, run the standard evaluation, to make sure you installed things correctly, see Listing 2.

3.1 Different LLMs

To get a feeling for how the standard agent performs on the different benchmarks with different LLMs, it is useful to run a few different LLMs. When you are using the API version, this requires acquiring different API keys, which may cost you money, or finding out which free tiers for students are possible. It is also recommended to run open LLMs, and see how they perform. Especially the open version from OpenAI, DeepSeek, OLMo and Qwen may work well.

Running these models is important, and will lead to more insight into the relative strengths of the various models.

3.2 Different Puzzles

Next, to get a better understanding of the benchmark, you should, after running the LLM agent in the different puzzle problems, have a closer look at the prompts and the answers of LLM. See if you can see where the LLM agent fails, where the weakness of the agent lies. By close inspection of the individual actions, and a deeper understanding of the puzzle, you might be able to think of a way to improve your agent.

3.3 Questions 1

Answer the following questions:

1. Provide scripts, prompts, and code in a zip file Ass2-1 with a README
2. Which LLMs did you try and why? Which are hosted in the cloud, which are open and locally hosted? Which benchmarks did you try and why?
3. Create a performance matrix of (LLM x Benchmark). What does the performance metric exactly measure?
4. Reflect:
    - What did not go according to plan? How did you solve this challenge?
    - Reflect on what the specific benchmarks measure, and what specific LLMs are good at.
    - Are LLMs good at reasoning? How do your results support this claim?
    - Think of ideas to implement in your own agent (next exercise) and how they address current problems.

4 Exercise 2: Implement Own Reasoning Agent

Now it is time to implement your own learning agent. Write down your ideas for improvement of the agent. Go to the agents directory in balrog, and edit the `custom.py` file. Listing 3 reproduces the code for Custom Agent from the BALROG readme. The code can be run with the command in Listing 4 (specifying the custom agent, and running gently with fewer workers. Of course, you can specify your own LLM of choice in the command.

You can start with the provided code, run it, see how it works. Then, you can try to make small improvements. Try making changes to the prompt for the LLM, and to the way in which the Python code handles the learning in the agent, either with the history prompt builder, or in the plan, or in the learning algorithm.

Are you trying improvements that work for all benchmarks, or benchmark-specific improvements?

Suggestions: Chain of thought, agentic memory, benchmark-specific tricks. But be careful to leave some of your ideas for the next Exercise: self-reflection.

4.1 Warning

So many benchmarks, so many LLMs, so many prompts, so many combinations. It is tempting to go for completeness, and try to run all possible combinations. You will never make the deadline if you do that. Do not go this route.

The learning goal is to get insight. Please choose an interesting research question and choose a few benchmarks, a few LLMs, to highlight your topic. (We prefer smart questions over wasting compute on boring questions.)

```python
from balrog.agents.base import BaseAgent
import re


class CustomAgent(BaseAgent):
    def __init__(self, client_factory, prompt_builder):
        super().__init__(client_factory, prompt_builder)
        self.client = client_factory()
        self.plan = None

    def act(self, obs, prev_action=None):
        if prev_action:
            self.prompt_builder.update_action(prev_action)
        self.prompt_builder.update_observation(obs)

        plan_text = f"Current Plan:\n{self.plan}\n" if self.plan else "You have no plan yet.\n"

        planning_instructions = """
Review the current plan above if present. Decide whether to continue with it or make changes.
If you make changes, provide the updated plan. Then, provide the next action to take.
You must output an action at every step.
Format your answer in the following way:
PLAN: <your updated plan if changed, or "No changes to the plan." if the current plan is good>
ACTION: <your next action>
        """.strip()

        messages = self.prompt_builder.get_prompt()
        if messages and messages[-1].role == "user":
            messages[-1].content += "\n\n" + plan_text + "\n" + planning_instructions

        response = self.client.generate(messages)

        # Extract the plan and action from the LLM's response
        plan, action = self._extract_plan_and_action(response.completion)

        # Update the internal plan if it has changed
        if plan != "No changes to the plan.":
            self.plan = plan

        # Save the plan in the response.reasoning field and the action in response.completion
        response = response._replace(reasoning=plan, completion=action)
        return response

    def _extract_plan_and_action(self, response_text):
        plan_match = re.search(r"PLAN:\s*(.*?)(?=\nACTION:|\Z)", response_text, re.IGNORECASE | re.DOTALL)
        action_match = re.search(r"ACTION:\s*(.*)", response_text, re.IGNORECASE | re.DOTALL)

        plan = plan_match.group(1).strip() if plan_match else "No changes to the plan."
        action = action_match.group(1).strip() if action_match else None

        return plan, action
```

Listing 3: Agent code in BALROG

```
python eval.py \
  agent.type=custom \
  agent.max_image_history=0 \
  agent.max_history=16 \
  eval.num_workers=64 \
  client.client_name=openai \
  client.model_id=gpt-4o-mini-2024-07-18
```

Listing 4: Running the Agent evaluation

4.2 Questions 2

Answer the following questions:

1. Provide scripts, prompts, and code in a zip file Ass2-2 with a README
2. Provide the performance matrix
3. Describe your idea in two sentences. Does your agent focus on specific (single benchmark) or generic (all benchmarks) improvements? Does your agent focus on improving a specific LLM, or all? For which benchmark/LLM did it work best?
4. Reflect:
   - Which LLMs did you try, which benchmarks, and why?
   - How many runs did you perform, statistically speaking? What does the concept *brittleness* mean to you?
   - Reflect on how your agent idea worked when implemented, at different LLMs and different benchmarks. What would you do differently next time if you had more time?
   - Is it difficult to implement an agent harness? How generic is your harness?
   - is Agentic LLM well suited for reasoning problems?

5 Exercise 3: Self-Reflection

In some sense, self-reflection is a holy grail of AI: an agent reasoning about it's own reasoning. In this exercise, we will extend the BALROG agent towards self-reflection, where the LLM is used in different roles, such as evaluation and action selection. Self-reflection, as any adaptive system that chases a target that is itself being optimized, can be prone to instability and oscillations (brittleness).

Self-reflection can be achieved primarily by the prompts (see, for example, ReAct [Yao et al., 2022]), or in the algorithmic harness (see, for example, Reflexion [Shinn et al., 2024]). Note that these works are already somewhat older, and the literature may contain newer ideas, such as the use of agentic memory, or

chunking, for self-reflection. Chapter 10 in the book contains some explanation of ReAct and Reflexion.

Choose a self-reflection approach, and implement it in BALROG. Suggestions: ReAct, Reflexion, ICRL, an agentic memory approach such as chunking, or another idea for the literature, or your own idea.

5.1 Questions 3

Answer the following questions:

1. Provide scripts, prompts, and code in a zip file Ass2-3 with a README
2. Provide the performance matrix
3. Describe your idea in two sentences. How does your approach self-reflect?
4. Which LLMs/Benchmarks are you focusing on? For which benchmark did it work best?
5. Reflect:
   - What did not go according to plan? What would you do different next time if you had more time?
   - How stable are your results and for which LLMs and benchmarks?
   - Is self-reflection well-suited for reasoning problems?
6. Bonus question, not for credit: philosophize in less than five sentences on how self-reflection in agentic LLMs is (not) different from self-reflection in humans.

6 Hints on Evaluation

The performance of your agent will be reported by BALROG as a progression percentage. Record the progression numbers for each benchmark, and for the agent types that you try. Do this in a structured way, you may want to write a Python or shell script to automate the running of the experiments and the logging of the results, perhaps also plotting the progression in a graph.

What agent types work well, and on what benchmark do they work well? Make a matrix in which you log progression for the agent types and the benchmark types that you try.

Note that LLM experiments are inherently non-deterministic: two runs with the same prompt may give different results. Therefore, perform multiple runs of each experiment. If you get five identical results in a row, you may start to trust these results. If they vary widely, you may want to investigate. Is the reason purely statistical? Does turning down the temperature solve the problem, or is there another problem?

Do not blindly trust the progression numbers that are reported. Since you are using code on a computer, there may be bugs, or other issues. The BALROG system facilitates delving deeper into the runs. It logs csv files that you can look at with a spreadsheet. Have a look at some of the runs, and decide if it makes sense what you see, or if the agents are exhibiting degenerative behavior. If they do, try to find the reason. Did you do something wrong? Is there unexpected interference between the benchmark and the agent behavior? Is the reward landscape too flat, so that there is no signal for the agent to go after, and all it does is random actions running in circles in a way that makes no sense at all? Is there something that you can do about this?

Please note that the field of large language models, and especially of agentic LLMs, is new, and our insights and our software frameworks have not yet stabilized. Some unexpected results are to be expected, and some challenges or frustration as well. You wanted excitement?

A structured way of experimentation and result-logging is your friend. Planning and organization can make the running, evaluation, and improvement of practical agentic LLM experiments an enjoyable exercise.

References

Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, et al. Balrog: Benchmarking agentic llm and vlm reasoning on games. *arXiv preprint arXiv:2411.13543*, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.